

LISTAS EM PYTHON

Conteúdo explicativo e exercícios de fixação

Prof. Esp Kelrison Coêlho

Componente: Linguagem de Programação (Python)

Curso: Técnico de Nível Médio em Informática

Objetivo: compreender como armazenar, acessar e manipular conjuntos de dados usando listas.

A lista será a ponte entre variáveis individuais e pequenos sistemas de cadastro. Depois deste conteúdo, os estudantes estarão preparados para usar **for** na leitura e manipulação de vários registros.

1. O que é uma lista?

Uma **lista** é uma estrutura de dados usada para armazenar vários valores dentro de uma única variável. Em Python, ela é representada por **colchetes []** e os elementos são separados por vírgulas.

```
nomes = ["Ana", "Carlos", "João"]
```

Nesse exemplo, a variável **nomes** guarda três valores. Em vez de criar uma variável para cada pessoa, podemos trabalhar com todos os nomes dentro de uma única lista.

Por que usar listas?

Listas são úteis quando temos um conjunto de informações: nomes de alunos, produtos, preços, notas, cidades, usuários etc.

Sem lista	Com lista
<pre>nome1 = "Ana" nome2 = "Carlos" nome3 = "João"</pre>	<pre>nomes = ["Ana", "Carlos", "João"]</pre>

Importante: uma lista pode ser alterada depois de criada. Por isso, dizemos que listas são **mutáveis**.

Exemplos:

```
idades = [16, 17, 18, 19]
precos = [10.50, 25.90, 8.00]
aprovados = [True, False, True]
```

Também é possível ter diferentes tipos de dados em uma lista, mas, para os primeiros exercícios, é recomendável manter dados com uma finalidade clara e consistente.

2. Posições e acesso aos elementos

Cada elemento ocupa uma posição chamada **índice**. Em Python, a contagem começa em **0**, e não em 1.

```
nomes = ["Ana", "Carlos", "João", "Maria"]  
# índice: 0 1 2 3
```

Índice	Valor
0	Ana
1	Carlos
2	João
3	Maria

Para acessar um elemento, usamos o índice entre colchetes:

```
print(nomes[0]) # Ana  
print(nomes[2]) # João
```

Também podemos usar índices negativos para começar pelo final:

```
print(nomes[-1]) # Maria  
print(nomes[-2]) # João
```

Atenção: acessar uma posição que não existe provoca um erro. Para conhecer o tamanho da lista, usamos **len()**.

```
print(len(nomes))
```

3. Alterando uma lista

Como a lista é mutável, podemos modificar um elemento informando sua posição.

```
nomes = ["Ana", "Carlos", "João"]
nomes[1] = "Pedro"
print(nomes)
# ["Ana", "Pedro", "João"]
```

Também podemos adicionar elementos.

```
nomes.append("Maria")
```

append() adiciona um elemento no final da lista.

Para inserir em uma posição específica, usamos **insert()**.

```
nomes.insert(1, "Lucas")
```

Para **remove**, podemos usar **remove()**, quando conhecemos o valor, ou **pop()**, quando queremos trabalhar com uma posição.

```
nomes.remove("Pedro")
nomes.pop(0)
```

Operação	Exemplo	O que faz?
Adicionar no final	<code>lista.append(valor)</code>	Acrescenta um elemento.
Inserir	<code>lista.insert(pos, valor)</code>	Coloca um elemento em determinada posição.
Remover por valor	<code>lista.remove(valor)</code>	Remove o primeiro elemento com aquele valor.
Remover por posição	<code>lista.pop(pos)</code>	Remove o elemento da posição indicada.
Quantidade	<code>len(lista)</code>	Retorna a quantidade de elementos.

4. Listas e o laço for

Uma das principais vantagens das listas é poder percorrer seus elementos. O **for** permite executar uma ação para cada item da lista.

```
nomes = ["Ana", "Carlos", "João"]

for nome in nomes:
    print(nome)
```

Leia o código como: *“para cada nome dentro de nomes, mostre o nome”*.

Esse padrão será muito importante para os próximos conteúdos de cadastro e CRUD.

Percorrendo números

```
idades = [16, 17, 18, 20]

for idade in idades:
    print(idade)
```

Podemos também fazer cálculos durante a repetição.

```
notas = [7, 8, 6, 9]

soma = 0

for nota in notas:
    soma = soma + nota

print(soma)
```

Outra possibilidade é usar enumerate() quando precisamos do índice e do valor.

```
for indice, nome in enumerate(nomes):
    print(indice, nome)
```

Neste momento, o mais importante é dominar o padrão básico **for item in lista**.

5. Outras operações úteis

Algumas operações aparecem com frequência quando trabalhamos com listas.

Operação	Exemplo	Finalidade
Verificar se existe	"Ana" in nomes	Retorna True ou False.
Contar ocorrências	notas.count(7)	Conta quantas vezes o valor aparece.
Encontrar posição	nomes.index("Ana")	Retorna o índice do valor.
Ordenar	nomes.sort()	Ordena a própria lista.
Copiar	nova = nomes.copy()	Cria uma cópia.

Exemplo de busca:

```
nomes = ["Ana", "Carlos", "João"]

nome = "Carlos"

if nome in nomes:
    print("Nome encontrado!")
else:
    print("Nome não encontrado.")
```

Boa prática: escolha nomes de variáveis que expliquem o conteúdo, como **nomes**, **idades** e **produtos**.

6. Exemplo prático: lista de produtos

Imagine uma pequena loja que precisa guardar os nomes dos produtos cadastrados.

```
produtos = ["Teclado", "Mouse", "Monitor", "Headset"]
```

Listando todos os produtos:

```
for produto in produtos:  
    print(produto)
```

Adicionando um novo produto:

```
produtos.append("Webcam")
```

Alterando um produto pela posição:

```
produtos[1] = "Mouse sem fio"
```

Procurando um produto:

```
if "Monitor" in produtos:  
    print("Produto encontrado!")
```

Removendo um produto:

```
produtos.remove("Headset")
```

A partir dessa ideia, podemos construir sistemas maiores, como cadastros de alunos, produtos ou funcionários.

Próximo passo: combinar listas + entrada de dados + condições + **for** para desenvolver pequenas aplicações de gerenciamento de informações.

7. Exercícios de fixação

Orientação: antes de programar, leia o contexto, identifique quais dados serão armazenados e defina as variáveis e os tipos de dados necessários.

Exercício 1 — Lista de notas

Uma professora deseja armazenar as notas de 5 estudantes em uma lista.

Faça um programa que:

1. Crie uma lista vazia chamada **notas**.
2. Leia 5 notas e adicione cada uma à lista usando **append()**.
3. Utilize um **for** para mostrar todas as notas cadastradas.
4. Mostre a quantidade de notas armazenadas.
5. Mostre a maior e a menor nota.

Antes de codificar: qual é o tipo de dado de cada nota? Por que a lista é mais adequada do que criar cinco variáveis diferentes?

Exercício 2 — Cadastro simples de produtos

Uma pequena loja precisa registrar 6 produtos. Para esta primeira versão, serão armazenados apenas os nomes.

Faça um programa que:

1. Crie uma lista vazia chamada **produtos**.
2. Leia o nome de 6 produtos e adicione-os à lista.
3. Utilize um **for** para exibir todos os produtos numerados.
4. Pergunte o nome de um produto para pesquisar.
5. Informe se o produto foi encontrado ou não.
6. Mostre quantos produtos estão cadastrados.

Desafio: permita que o usuário remova um produto informado por ele.

8. Resumo para consulta rápida

Conceito	Exemplo
Criar lista	<code>nomes = ["Ana", "Carlos"]</code>
Acessar posição	<code>nomes[0]</code>
Alterar	<code>nomes[0] = "Maria"</code>
Adicionar	<code>nomes.append("João")</code>
Inserir	<code>nomes.insert(1, "Pedro")</code>
Remover por valor	<code>nomes.remove("Pedro")</code>
Remover por posição	<code>nomes.pop(0)</code>
Quantidade	<code>len(nomes)</code>
Percorrer	<code>for nome in nomes:</code>
Verificar existência	<code>"Ana" in nomes</code>

Pontos essenciais

- Lista guarda vários valores em uma única variável.
- Os índices começam em 0.
- Listas podem ser alteradas depois de criadas.
- **append()**, **insert()**, **remove()** e **pop()** são operações fundamentais.
- **len()** informa a quantidade de elementos.
- **for** permite percorrer os elementos da lista.

Material didático introdutório — foco em compreensão, prática e preparação para atividades de cadastro com laços de repetição.